MELBOURNE REAL ESTATE PRICE PREDICTION

PROJECT DOCUMENTATION

Name: AASIR JAFFER LONE

Registration No.: 25BAI10844

Programme: B.Tech CSE (AI/ML)

College: VIT Bhopal University

Subject: Fundamentals in AI/ML

Faculty Guide: Bandla Pawan Babu

Abstract

This project implements and evaluates machine learning models to predict residential property prices in Melbourne, Australia, using the publicly available Melbourne Housing dataset. The workflow covers data loading, preprocessing, automatic numerical feature selection, model training, hyperparameter tuning, and performance evaluation. A baseline Decision Tree Regressor is compared against an optimized Decision Tree and a Random Forest Regressor, with Random Forest selected as the final model based on its lower validation Mean Absolute Error (MAE) and higher coefficient of determination (R^2). The final pipeline provides a reusable, configurable template for tabular regression problems in real estate analytics.

1. Introduction

Real estate price prediction is a key application of machine learning in the domain of property valuation and urban analytics. Accurate estimates of house prices help buyers, sellers, and policy makers make informed decisions based on historical market data and property characteristics.

In this project, we build a supervised regression pipeline using the Melbourne Housing dataset. The goal is to learn the relationship between numerical property attributes (such as number of rooms, distance from the central business district, land size, and building area) and the target variable, the sale price. The project is developed as part of the course 'Fundamentals in AI/ML' and is implemented in Python using standard open-source libraries.

2. Problem Statement and Objectives

Problem Statement

Given a historical dataset of property sales in Melbourne, predict the sale price of a house using its numerical features. The model should generalize well to unseen data and provide interpretable insights into the factors that most strongly influence price.

Objectives

- Load and explore the Melbourne Housing dataset and identify relevant numerical features.
- Handle missing values and construct a clean feature matrix and target vector for modelling.
- Train a baseline Decision Tree Regressor and measure its validation performance.
- Perform hyperparameter tuning on the Decision Tree (max_leaf_nodes) to reduce overfitting and improve MAE.
- Train a Random Forest Regressor, compare it against the Decision Tree models, and select the best model based on MAE and R^2.
- Analyze feature importance to understand which variables contribute most to price prediction.

3. Dataset Description

The project uses the "Melbourne Housing" dataset (melb_data.csv), which contains historical sales records for residential properties in Melbourne. Each row represents a single property transaction.

Key properties of the dataset:

- Total samples (after cleaning): 6,830.
- Total original columns: 21, including both categorical and numerical attributes.
- Target variable: Price (continuous, sale price in Australian dollars).
- Numerical feature columns used after auto-selection: Rooms, Distance, Postcode, Bedroom2, Bathroom, Car, Landsize, BuildingArea, YearBuilt, Lattitude, Longtitude, Propertycount.

The notebook automatically detects all numeric columns except the target as features when feature_columns is set to 'auto'. Only rows with no missing values in the selected numeric features and non-missing target values are retained for training and validation.

4. Tech Stack

- Programming Language: Python 3.x
- Environment: Jupyter Notebook / Python script (MODEL-1.py)
- Data Manipulation: pandas, numpy
- Machine Learning: scikit-learn (DecisionTreeRegressor, RandomForestRegressor, train_test_split, cross_val_score, metrics)
- Visualization: matplotlib (feature importance and cross-validation plots)

5. Methodology and Model Architecture

5.1 Overall Workflow

The machine learning pipeline is implemented in a modular fashion, making it easy to adapt to similar tabular regression problems. The major steps are:

- Configuration of file path (csv_path), target column (target_col), and feature selection mode (feature_columns = 'auto').
- Loading the CSV into a pandas DataFrame and printing basic information such as shape, column names, data types, and missing values.
- Automatic detection of numeric feature columns (excluding the target), construction of the feature matrix X and target vector y, and dropping rows with missing feature or target values.
- Splitting the cleaned data into training and validation sets using an 80/20 split with a fixed random_state for reproducibility.
- Training and evaluating a baseline Decision Tree Regressor.
- Hyperparameter tuning of the Decision Tree via a grid of max_leaf_nodes values.
- Training and evaluating a Random Forest Regressor, followed by cross-validation over different numbers of trees.
- Extracting feature importance scores from the Random Forest model.

5.2 Data Preprocessing

- Verify that the specified target column exists in the dataset; raise an error if not.
- Auto-detect all numeric feature columns except the target when feature_columns = 'auto'.
- Check for missing values in selected features; drop rows with missing feature values to obtain a complete feature matrix.
- Ensure there are no missing target values by filtering y based on the valid feature indices.

- Final dataset after preprocessing: 6,830 rows × 12 numeric features.

5.3 Train–Validation Split

The cleaned dataset is split into training and validation sets using train_test_split with train_size = 0.8, test_size = 0.2, and random_state = 1 for reproducibility. This results in approximately 5,464 training samples and 1,366 validation samples.

5.4 Model 1 – Decision Tree Regressor (Baseline)

- A basic DecisionTreeRegressor is trained on the training set without hyperparameter tuning.
- Predictions are made on the validation set and Mean Absolute Error (MAE) is computed.
- The baseline Decision Tree achieves a validation MAE of approximately 229,232.

5.5 Model 1 (Optimized) – Decision Tree with Tuned max_leaf_nodes

- A small grid search is performed over max_leaf_nodes values: {5, 25, 50, 100, 250, 500}.
- For each candidate value, a Decision Tree is trained on the training data and evaluated on the validation set using MAE.
- The best tree size is selected based on the lowest validation MAE; max_leaf_nodes = 500 is chosen with MAE ≈ 208,956.
- A final Decision Tree with max_leaf_nodes = 500 is then trained on the full cleaned dataset (X, y).

5.6 Model 2 – Random Forest Regressor (Final Model)

- A RandomForestRegressor with n_estimators = 100 and random_state = 1 is trained on the training set.
- Validation predictions are generated and MAE, Root Mean Squared Error (RMSE), and R^2 score are computed.
- The Random Forest model achieves a significantly lower validation MAE of about 173,738, RMSE ≈ 332,026, and $R^2 \approx 0.794$.
- Random Forest reduces overfitting compared to the single Decision Tree and offers more stable generalization.

5.7 Cross-Validation and Hyperparameter Tuning for Random Forest

- To analyze the effect of the number of trees, 3-fold cross-validation is performed for n_estimators ∈ {50, 100, 150, 200, 250, 300, 350, 400}, using negative MAE as the scoring metric.
- The average cross-validated MAE is reported for each value. The best region is around 150 trees, with MAE ≈ 211,795.
- A line plot of n_estimators vs. MAE is generated to visualize the trade-off between model size and performance.

5.8 Feature Importance Analysis

Feature importance scores from the trained Random Forest model indicate how much each feature contributes to reducing prediction error. The top three most important features are:

- BuildingArea – importance ≈ 0.375 (most influential feature).
- Distance – importance ≈ 0.156.
- Postcode – importance ≈ 0.110.

Other influential features include Landsize, YearBuilt, Lattitude, Longtitude, Rooms, Propertycount, Bathroom, Car, and Bedroom2, with smaller but non-negligible contributions.

6. Model Performance and Results

6.1 Evaluation Metrics

The following regression metrics are used to evaluate model performance on the validation set:

- Mean Absolute Error (MAE): average absolute difference between predicted and actual prices.
- Root Mean Squared Error (RMSE): square root of the average squared difference; penalizes large errors more heavily.
- R^2 Score (coefficient of determination): proportion of variance in the target explained by the model (closer to 1 is better).

6.2 Comparison of Models

Model	Description	Validation MAE	R^2 Score
Decision Tree (Baseline)	Default DecisionTreeRegressor	≈ 229,232	–
Decision Tree (Tuned)	max_leaf_nodes = 500	≈ 208,956	–
Random Forest	100 trees, default settings	≈ 173,738	≈ 0.794

Random Forest clearly outperforms both the baseline and tuned Decision Tree models in terms of validation MAE and R^2. It reduces the error by roughly 24% relative to the basic Decision Tree and captures a substantial proportion of the variance in prices.

6.3 Sample Predictions

Sample predictions on the validation set show that the Random Forest model can approximate actual sale prices reasonably well, though some properties with very high or unusual prices still incur larger percentage errors. For a typical subset of 10 validation examples, absolute percentage errors mostly fall within a range of roughly ±40%, with smaller errors for mid-range properties.

7. How to Clone and Run the Project

7.1 Prerequisites

- Python 3.x installed on your system.
- pip package manager.
- Git (optional, for cloning via command line).

7.2 Clone the Repository

Using Git:

1. Open a terminal or command prompt.

2. Run the following commands:

```
git clone https://github.com/your-username/real-estate-prediction.git

cd real-estate-prediction
```

If you have the project as a ZIP file, simply extract it and navigate into the project directory.

7.3 Install Dependencies

Install the required Python libraries using pip:

```
pip install pandas numpy scikit-learn matplotlib jupyter
```

7.4 Project Structure

- Real-Estate-Price-Prediction-model.ipynb – main Jupyter Notebook with the complete ML workflow.
- MODEL-1.py – Python script version of the model pipeline.
- melb_data.csv – Melbourne Housing dataset.
- README.md – High-level project overview and usage instructions.

7.5 Running the Notebook

1. Start Jupyter Notebook in the project directory:

```
jupyter notebook
```

2. Open 'Real-Estate-Price-Prediction-model.ipynb'.

3. Run all cells sequentially. Ensure that csv_path is set to 'melb_data.csv' (or the correct relative path) and target_col is set to 'Price'.

7.6 Running the Python Script

From the project directory, run:

```
python MODEL-1.py
```

The script will load the dataset, preprocess the data, train the models, print evaluation metrics, and summarize the best model along with feature importance.

8. Limitations and Future Work

Limitations

- Only numerical features are used; categorical attributes such as suburb, type, and method are not encoded or leveraged.
- Missing values are handled by dropping rows, which reduces the dataset size and may bias the model.
- The model is trained and evaluated on a single train/validation split; no full cross-validation or test set is used for final reporting.

Future Work

- Incorporate categorical features using techniques such as one-hot encoding or target encoding.
- Experiment with additional models such as Gradient Boosting, XGBoost, or LightGBM for potentially higher accuracy.
- Implement full k-fold cross-validation and a separate held-out test set for more robust performance estimates.
- Perform more advanced feature engineering (e.g., interaction features, log-transforms of skewed variables).
- Deploy the trained model as a REST API or simple web application for interactive predictions.

9. Conclusion

This project demonstrates an end-to-end machine learning pipeline for predicting house prices using the Melbourne Housing dataset. Starting from raw CSV data, we cleaned and prepared the dataset, trained and tuned multiple tree-based regression models, and selected a Random Forest Regressor as the final model based on superior validation MAE and R^2. Feature importance analysis highlighted building area, distance to the city, and postcode as the dominant drivers of price. The modular, configurable architecture makes

this codebase a solid starting point for more advanced experiments and real-world deployment in the domain of real estate analytics.

(End of Document)